

# STYLECART-FASHION E-COMMERCE DATABASE MANAGEMENT SYSTEM

## PRODUCT REVIEW AND RATING MANAGEMENT SYSTEM

### 1.Database Design

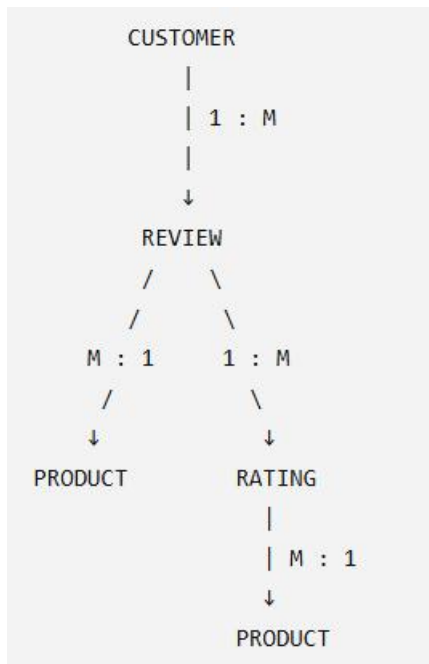
#### 1.1 Review Table

| Attribute      | Data Type     | Key / Constraint |
|----------------|---------------|------------------|
| Review_ID      | NUMBER        | Primary Key      |
| Customer_ID    | NUMBER        | Foreign Key      |
| Product_ID     | NUMBER        | Foreign Key      |
| Rating         | NUMBER(1)     | NOT NULL         |
| Review_Comment | VARCHAR2(500) | NOT NULL         |
| Review_Date    | DATE          | NOT NULL         |

#### 1.2 Rating Table

| Attribute    | Data Type | Key / Constraint |
|--------------|-----------|------------------|
| Rating_ID    | NUMBER    | Primary Key      |
| Review_ID    | NUMBER    | Foreign Key      |
| Product_ID   | NUMBER    | Foreign Key      |
| Rating_Value | NUMBER(1) | NOT NULL         |

### 2.Table Relationships



### Relationship Explanation

- 1. Customer → Review**  
One customer can write many reviews.  
Review.Customer\_ID references Customer.Customer\_ID.
- 2. Product → Review**  
One product can have many reviews.  
Review.Product\_ID references Product.Product\_ID.
- 3. Review → Rating**  
Each review has a corresponding rating record.  
Rating.Review\_ID references Review.Review\_ID.
- 4. Product → Rating**  
One product can have many rating records.  
Rating.Product\_ID references Product.Product\_ID.

### 3. Table Creation

#### 3.1 Create Review Table

```
SQL> CREATE TABLE Review (  
2     Review_ID NUMBER PRIMARY KEY,  
3     Customer_ID NUMBER NOT NULL,  
4     Product_ID NUMBER NOT NULL,  
5     Rating NUMBER(1) NOT NULL,  
6     Review_Comment VARCHAR2(500) NOT NULL,  
7     Review_Date DATE NOT NULL,  
8     FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID),  
9     FOREIGN KEY (Product_ID) REFERENCES Product(Product_ID)  
10 );
```

Table created.

### 3.2 Create Rating Table

```
SQL> CREATE TABLE Rating (  
2     Rating_ID NUMBER PRIMARY KEY,  
3     Review_ID NUMBER NOT NULL,  
4     Product_ID NUMBER NOT NULL,  
5     Rating_Value NUMBER(1) NOT NULL,  
6     FOREIGN KEY (Review_ID) REFERENCES Review(Review_ID),  
7     FOREIGN KEY (Product_ID) REFERENCES Product(Product_ID)  
8 );
```

Table created.

### 4.Review Data Insertion

```

SQL> INSERT INTO Review
  2 (Review_ID, Customer_ID, Product_ID, Rating, Review_Comment, Review_Date)
  3 VALUES
  4 (601, 1, 1, 5, 'Good quality product', SYSDATE);

1 row created.

SQL>
SQL> INSERT INTO Review
  2 (Review_ID, Customer_ID, Product_ID, Rating, Review_Comment, Review_Date)
  3 VALUES
  4 (602, 2, 2, 4, 'Very comfortable and stylish', SYSDATE);

1 row created.

SQL>
SQL> INSERT INTO Review
  2 (Review_ID, Customer_ID, Product_ID, Rating, Review_Comment, Review_Date)
  3 VALUES
  4 (603, 3, 3, 5, 'Nice product and good quality', SYSDATE);

1 row created.

SQL>
SQL> INSERT INTO Review
  2 (Review_ID, Customer_ID, Product_ID, Rating, Review_Comment, Review_Date)
  3 VALUES
  4 (604, 1, 4, 3, 'Average quality', SYSDATE);

1 row created.

SQL>
SQL> INSERT INTO Review
  2 (Review_ID, Customer_ID, Product_ID, Rating, Review_Comment, Review_Date)

```

## 5. Rating Data Insertion

```
SQL> INSERT INTO Rating
  2 (Rating_ID, Review_ID, Product_ID, Rating_Value)
  3 VALUES (701, 601, 1, 5);
```

1 row created.

```
SQL>
SQL> INSERT INTO Rating
  2 (Rating_ID, Review_ID, Product_ID, Rating_Value)
  3 VALUES (702, 602, 2, 4);
```

1 row created.

```
SQL>
SQL> INSERT INTO Rating
  2 (Rating_ID, Review_ID, Product_ID, Rating_Value)
  3 VALUES (703, 603, 3, 5);
```

1 row created.

```
SQL>
SQL> INSERT INTO Rating
  2 (Rating_ID, Review_ID, Product_ID, Rating_Value)
  3 VALUES (704, 604, 4, 3);
```

1 row created.

```
SQL>
SQL> INSERT INTO Rating
  2 (Rating_ID, Review_ID, Product_ID, Rating_Value)
  3 VALUES (705, 605, 1, 4);
```

1 row created.

## 6.Retrieve Product Review Details

```

SQL> SELECT
2      r.Review_ID,
3      r.Customer_ID,
4      r.Product_ID,
5      p.Product_Name,
6      r.Rating,
7      r.Review_Comment,
8      r.Review_Date
9  FROM Review r
10 JOIN Product p
11 ON r.Product_ID = p.Product_ID
12 ORDER BY r.Product_ID;

```

```

REVIEW_ID CUSTOMER_ID PRODUCT_ID
-----

```

```

PRODUCT_NAME
-----

```

```

RATING
-----

```

```

REVIEW_COMMENT
-----

```

```

REVIEW_DA
-----

```

```

          601          1          1
Lakme Foundation
          5

```

```

REVIEW_ID CUSTOMER_ID PRODUCT_ID
-----

```

```

PRODUCT_NAME
-----

```

```

RATING
-----

```

```

REVIEW_COMMENT
-----

```

## 7. Calculate Average Product Ratings

```

SQL> SELECT
  2     p.Product_ID,
  3     p.Product_Name,
  4     COUNT(r.Review_ID) AS Total_Reviews,
  5     ROUND(AVG(r.Rating), 2) AS Average_Rating
  6 FROM Product p
  7 JOIN Review r
  8 ON p.Product_ID = r.Product_ID
  9 GROUP BY p.Product_ID, p.Product_Name
 10 ORDER BY Average_Rating DESC;

```

PRODUCT\_ID

-----

PRODUCT\_NAME

-----

TOTAL\_REVIEWS AVERAGE\_RATING

-----

|                |   |
|----------------|---|
| 3              |   |
| Sugar Lipstick |   |
| 1              | 5 |

|                  |     |
|------------------|-----|
| 1                |     |
| Lakme Foundation |     |
| 2                | 4.5 |

PRODUCT\_ID

-----

PRODUCT\_NAME

-----

TOTAL\_REVIEWS AVERAGE\_RATING

-----

|                    |   |
|--------------------|---|
| 2                  |   |
| Maybelline Mascara |   |
| 1                  | 4 |

## 8. Identify Highly Rated Products

```

SQL> SELECT
  2     p.Product_ID,
  3     p.Product_Name,
  4     ROUND(AVG(r.Rating), 2) AS Average_Rating
  5 FROM Product p
  6 JOIN Review r
  7 ON p.Product_ID = r.Product_ID
  8 GROUP BY p.Product_ID, p.Product_Name
  9 HAVING AVG(r.Rating) >= 4
 10 ORDER BY Average_Rating DESC;

```

PRODUCT\_ID

-----

PRODUCT\_NAME

-----

AVERAGE\_RATING

-----

|                |   |
|----------------|---|
| 3              |   |
| Sugar Lipstick | 5 |

|                  |     |
|------------------|-----|
| 1                |     |
| Lakme Foundation | 4.5 |

PRODUCT\_ID

-----

PRODUCT\_NAME

-----

AVERAGE\_RATING

-----

|                    |   |
|--------------------|---|
| 2                  |   |
| Maybelline Mascara | 4 |

## 9.Product Rating Report

```

SQL> SELECT
  2     p.Product_ID,
  3     p.Product_Name,
  4     COUNT(r.Review_ID) AS Total_Reviews,
  5     ROUND(AVG(r.Rating), 2) AS Average_Rating,
  6     MAX(r.Rating) AS Highest_Rating,
  7     MIN(r.Rating) AS Lowest_Rating
  8 FROM Product p
  9 JOIN Review r
 10 ON p.Product_ID = r.Product_ID
 11 GROUP BY p.Product_ID, p.Product_Name
 12 ORDER BY Average_Rating DESC;

```

PRODUCT\_ID

PRODUCT\_NAME

TOTAL\_REVIEWS AVERAGE\_RATING HIGHEST\_RATING LOWEST\_RATING

|                  |   |     |   |   |
|------------------|---|-----|---|---|
| 3                |   |     |   |   |
| Sugar Lipstick   | 1 | 5   | 5 | 5 |
| 1                |   |     |   |   |
| Lakme Foundation | 2 | 4.5 | 5 | 4 |

PRODUCT\_ID

PRODUCT\_NAME

TOTAL\_REVIEWS AVERAGE\_RATING HIGHEST\_RATING LOWEST\_RATING

2